

EduVerify — Azure-Only Deployment Plan

Document: Full Microsoft Azure deployment with Azure Blob Storage (no Cloudinary)

Application: EduVerify Pre-Verification Portal · Node.js 20 + PostgreSQL

Scale target: ~5,000 students per admission cycle (scalable to 25,000+)

Prepared: June 2026

Yes — this is possible. EduVerify can run entirely on Azure with student documents stored in **Azure Blob Storage** instead of Cloudinary. The application already keeps file metadata in PostgreSQL (`file_public_id`, `file_name`, `file_format`) and streams previews through the API. Replacing the Cloudinary module with an Azure Blob adapter is a **focused code change** (~1–2 days development + testing), not a full rewrite.

1. Current vs target architecture

Layer	Today (Render)	Target (Azure-only)
Web + API	Render Web Service	Azure App Service (Linux, Node 20)
Database	Render PostgreSQL	Azure Database for PostgreSQL Flexible Server
Document files	Cloudinary (authenticated assets)	Azure Blob Storage (private container + SAS)
Secrets	Platform env vars	Azure Key Vault + Managed Identity
HTTPS / domain	*.onrender.com	Custom domain + App Service managed certificate
Monitoring	Render logs	Application Insights + Log Analytics

CDN (optional)	Cloudinary CDN	Azure Front Door or Blob static website (not needed — files are private)
----------------	----------------	--

2. Azure-only architecture diagram

```
[ Students · Verification Staff · Campus network ] | Azure DNS (eduverify.college.edu) | Azure Front Door (optional: WAF, TLS, geo routing) | Azure App Service – Linux, Node.js 20 (Managed Identity – no storage keys in code) | | | +---- Azure Key Vault | JWT_SECRET, SEED_ADMIN_PASSWORD | +-----+-----+ | | Azure Database for PostgreSQL Azure Storage Account Flexible Server (private/VNet) Container: eduverify-documents - students, documents metadata Access: Private (no public blob) - verify schedule, audit logs Encryption: Microsoft-managed keys Lifecycle: Cool/Archive after 2 years
```

3. Why Azure Blob replaces Cloudinary cleanly

Cloudinary is used in four ways in EduVerify. Each maps directly to Azure Blob:

Cloudinary function	Used for	Azure Blob equivalent
<code>uploadBuffer()</code>	Student document upload	<code>BlockBlobClient.upload()</code> to path <code>{appNo}/{docCode}_{timestamp}.pdf</code>
<code>signedUrl()</code>	Time-limited download link	User delegation SAS or account SAS (1-hour expiry, read-only)
<code>fetchAssetBuffer()</code>	Admin preview + ZIP export	<code>downloadToBuffer()</code> via SDK (server-side, no public URL)
<code>destroyAsset()</code>	Re-upload / student delete	<code>deleteBlob()</code>

The database column `file_public_id` today stores Cloudinary public IDs. After migration it stores the **blob path** (e.g. `CSE2026001/AADHAAR_1719234567890.pdf`). No schema change is required.

Effort note: A new file `src/config/blobStorage.js` replaces `src/config/cloudinary.js`. Four files import storage today: `student.js`, `admin.js`, `docStream.js`, `deleteStudent.js`, plus `docs.js` for signed URLs. CSP headers in `server.js` remove Cloudinary domains. The `cloudinary` npm package is removed; `@azure/storage-blob` and `@azure/identity` are added.

4. Azure Blob Storage design

4.1 Storage account settings

Setting	Value
Performance	Standard (documents are not latency-critical)
Redundancy	LRS for pilot; ZRS or GRS for production
Access tier	Hot during admission season; lifecycle rule → Cool after 12 months
Public access	Disabled (container private)
Encryption	Microsoft-managed keys (or CMK in Key Vault if policy requires)
Soft delete	Enabled (7–14 days) — protects against accidental deletion
Versioning	Optional — useful when students re-upload documents
Container name	<code>eduverify-documents</code>
Blob path pattern	<code>{appNo}/{docCode}_{unixMs}.{ext}</code>

4.2 Security model (equivalent to Cloudinary “authenticated”)

- Container has **no anonymous read** — blobs are never world-readable.
- Students and staff never receive permanent blob URLs. The API generates **short-lived SAS tokens** (e.g. 60 minutes) for download links, same as today.

- Previews in the admin modal use **same-origin API streaming** (/api/.../preview) — already implemented; only the backend fetch source changes from Cloudinary to Blob.
- App Service uses **Managed Identity** with role Storage Blob Data Contributor on the container — no account keys in environment variables.
- Network: optional **Private Endpoint** for Storage so blob traffic stays on Azure backbone.

4.3 Environment variables (Azure-only — no Cloudinary)

Variable	Example / purpose
STORAGE_PROVIDER	azure (or remove Cloudinary vars entirely)
AZURE_STORAGE_ACCOUNT_NAME	steduverifyprod
AZURE_STORAGE_CONTAINER	eduverify-documents
AZURE_STORAGE_SAS_EXPIRY_MINUTES	60 (download link lifetime)
AZURE_CLIENT_ID	Only if using user-assigned managed identity
Remove: CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET	

With Managed Identity, DefaultAzureCredential authenticates automatically on App Service — no storage account key needed.

5. Code migration outline

5.1 New storage module (pseudocode)

src/config/blobStorage.js - isConfigured() → checks AZURE_STORAGE_ACCOUNT_NAME + container - uploadBuffer(buf, appNo, docCode) → returns { publicId: blobPath, format, resourceType } - signedUrl(doc) → generateBlobSASUrl(blobPath, read, 60 min) - fetchAssetBuffer(doc) → download blob to Buffer - destroyAsset(publicId) → deleteBlob(publicId) src/config/storage.js (thin facade) - exports functions from blobStorage OR cloudinary based on STORAGE_PROVIDER - allows phased cutover during migration

5.2 Files to update

File	Change
package.json	Add @azure/storage-blob, @azure/identity; remove cloudinary
src/config/blobStorage.js	New — core Blob implementation
src/config/cloudinary.js	Remove after cutover (or keep behind feature flag briefly)
src/routes/student.js	Import from storage.js; update error messages
src/lib/docStream.js	No UI change — fetch from Blob
src/lib/deleteStudent.js	Delete blobs instead of Cloudinary assets
server.js	CSP: remove res.cloudinary.com; keep blob: for previews
.env.example	Document Azure storage vars; remove Cloudinary section

5.3 Estimated development effort

Task	Duration
Implement blobStorage.js + facade	4–6 hours
Wire routes + test upload/preview/ZIP/delete	4–6 hours

Azure infra (storage account, MI, RBAC)	2–4 hours
Migrate existing Cloudinary files to Blob (if any production data)	4–8 hours (scripted)
UAT with 50–100 test students	1–2 days
Total	~3–5 working days

6. Azure resources checklist

Resource group: rg-eduverify-prod (region: **Central India** or institution-approved)

1. **App Service Plan** — Linux, Premium v3 P1v3 (production)
2. **Web App** — eduverify-prod, Node 20 LTS, Always On enabled
3. **PostgreSQL Flexible Server** — General Purpose 2 vCore, 32–64 GB, SSL required
4. **Storage Account** — steduverifyprod, private container eduverify-documents
5. **Key Vault** — JWT_SECRET, SEED_ADMIN_PASSWORD, DATABASE_URL (or use App Service connection strings)
6. **Managed Identity** — System-assigned on Web App; grant Storage Blob Data Contributor
7. **Application Insights** — linked to Web App
8. **Log Analytics Workspace** — central logging
9. **Private Endpoints** (optional) — PostgreSQL + Storage for campus-only access
10. **Azure Front Door** (optional) — WAF, DDoS, custom domain

7. Capacity — 5,000 students

Resource	Estimate	Recommendation
PostgreSQL data	2–8 GB metadata	GP 2 vCore, 32 GB storage, 7-day backup
Blob storage	50–150 GB (one cycle)	Standard ZRS; lifecycle to Cool after 12 months
Peak concurrent users	200–500	App Service scale 1→3 instances
Upload throughput	~5 MB × thousands of files	Blob handles easily; rate limit already in app (60/hr/IP)

8. Deployment & CI/CD

GitHub (main) → GitHub Actions 1. `npm ci --omit=dev` 2. Run smoke tests (optional) 3. Deploy zip to App Service staging slot 4. `GET /api/health` 5. Swap staging → production Startup (first deploy): `NODE_ENV=production AUTO_SETUP=true` → runs schema + admin seed `DATABASE_URL` from Key Vault reference

9. Migration from Render + Cloudinary

Step	Action
1	Provision Azure resources (App Service, PostgreSQL, Storage, Key Vault)
2	Deploy EduVerify with Blob storage code + Managed Identity
3	<code>pg_dump</code> from Render PostgreSQL → restore to Azure PostgreSQL
4	Run migration script: for each document with Cloudinary <code>file_public_id</code> , download via Cloudinary API and upload to Blob; update <code>file_public_id</code> to blob path

5	Verify sample students: preview, download, ZIP export, re-upload, delete
6	Point DNS to Azure; set CORS_ORIGIN to new URL
7	Decommission Render and Cloudinary after 30-day parallel run (optional)

Greenfield deploy (no existing uploads): Skip step 4 — deploy Blob code directly; no Cloudinary account needed.

10. Security & compliance

- All student PII and documents remain in chosen Azure region (data residency).
- PostgreSQL: SSL, firewall / VNet, least-privilege DB user.
- Blobs: private container, encryption at rest, soft delete, audit logs via Storage Analytics.
- No third-party document CDN — reduces vendor risk and DPA scope.
- Key Vault for secrets; Managed Identity eliminates storage keys in config.
- Existing app controls retained: JWT auth, rate limits, staff roles, audit log.

11. Estimated monthly cost (India region, indicative)

Service	Production (~5K students)
---------	---------------------------

App Service P1v3 (1 instance)	■8,000 – ■12,000
PostgreSQL GP 2 vCore	■10,000 – ■18,000
Blob Storage ~100 GB Hot	■1,500 – ■3,000
Key Vault + App Insights	■1,000 – ■3,000
Total (no Cloudinary fee)	■20,000 – ■36,000 / month

Blob-only storage is typically **lower cost** than Cloudinary at scale because you pay only for GB stored and egress, not per-transformation API.

12. Rollout timeline

Phase	Duration	Deliverables
A — Azure foundation	Week 1	RG, App Service, PostgreSQL, Storage, Key Vault, SSL domain
B — Blob code + deploy	Week 1–2	Replace Cloudinary module; staging UAT
C — Data migration	Week 2	DB import; optional file migration script
D — Load test	Week 2–3	5K bulk upload, concurrent document uploads
E — Go-live	Admission season	DNS cutover, monitoring, support runbook

13. Decision summary

Question	Answer
Can we deploy on Azure only?	Yes
Can we avoid Cloudinary?	Yes — use Azure Blob Storage
Major rewrite needed?	No — storage adapter swap (~5 files)
Recommended hosting	Azure App Service + PostgreSQL Flexible Server + Blob
Auth to Blob	Managed Identity (no keys in app)
Student download security	Private blobs + server-generated SAS (1 hour)
Timeline	3–5 dev days + 2–3 weeks infra/UAT

14. Prerequisites from college IT

1. Azure subscription (EA, CSP, or Pay-As-You-Go)
2. Approval for Azure region and data retention policy
3. DNS hostname (e.g. `eduverify.manipal.edu`)
4. Decision to proceed with Blob-only (this document)
5. Optional: VNet / Private Endpoint requirements
6. Maintenance window for Render cutover

